

Lab 3D: Capstone — Multi-Agent Build + Headless CI Check

Duration: 35 min **After:** Scale, Cost & New Surfaces lecture (Day 3, Block 4) **Prerequisites:** Labs 3A–3C complete (MCP server, subagents, ci-review script)

Objective

Run everything at once: a multi-agent build of a real feature on your toolkit repo, graded against a rubric, validated by your headless CI check. Success is binary: **toolkit repo complete + CI run green**.

Deliverable (toolkit chain 12 of 12 — FINAL)

The finished personal toolkit repo containing all five artifacts — **CLAUDE.md**, **skill**, **hook**, **command**, **plugin manifest** — plus a new feature built by agents and a green CI run proving it.

START MARKER: `cd business-dashboard` ; *clean git status; commit a known-good baseline. Create the capstone worktree inside the project's canonical worktree area and open it as its own VS Code window/session.*

Recovery boundary: *Git, not checkpoint rewind, is the durable boundary for Bash, subagent, and external changes.*

Fell behind? Minimum viable start:

```
Set up this project with: CLAUDE.md conventions, an Express server with src/routes/, a dashboard-code-review skill in .claude/skills/, a PostToolUse prettier hook in .claude/settings.json, a /dashboard-review command, subagents security-auditor and grader in .claude/agents/, and scripts/ci-review.sh that runs authenticated claude -p with --strict-mcp-config --disable-slash-commands --setting-sources user --tools "Read,Grep,Glob" --allowedTools "Read,Grep,Glob" --permission-mode dontAsk --max-turns 10 and fails below score 6.
```

Phase 1: Toolkit Inventory Gate (5 min)

Before building, verify all five toolkit artifacts exist. Start `claude` and run:

```
Verify this repo contains all five toolkit artifacts and report PRESENT/MISSING for each:
1. CLAUDE.md (root, under 200 lines)
2. A skill: .claude/skills/dashboard-code-review/SKILL.md with name/description/disable-model-i
nvocation/allowed-tools frontmatter
3. A hook: .claude/hooks/post-edit-format.sh registered as PostToolUse in .claude/settings.json
4. A human-invoked skill: .claude/skills/dashboard-review/SKILL.md using $ARGUMENTS with `disab
le-model-invocation: true`
5. A plugin manifest: ../team-marketplace/plugins/dashboard-toolkit/.claude-plugin/plugin.json
(or plugin installed from team-marketplace)
```

Expected output marker: five PRESENT lines. Fix any MISSING with the relevant lab's recovery prompt before continuing — the capstone grade depends on it.

Phase 2: Plan the Feature in VS Code (5 min)

Shift+Tab to Plan mode, then:

```
Plan a "Weekly Report" feature:
- GET /api/report/weekly - JSON summary: metrics by category, task completion rate, top 3 high-
priority open tasks
- A "Weekly Report" section on the dashboard UI rendering that data
- Data shapes should match the dashboard-metrics MCP server where relevant
- Must follow CLAUDE.md conventions and pass our rubric.md criteria
Produce a step-by-step plan with file list and test strategy. Save it to IMPLEMENTATION_PLAN.md
when I approve.
```

Review the Markdown plan in VS Code, add at least one inline comment that sharpens scope or verification, iterate once, approve, and let it save the plan.

Expected output marker: `IMPLEMENTATION_PLAN.md` exists; no source files changed yet (Plan mode held the line).

Phase 3: Multi-Agent Build (12 min)

Shift+Tab back to Default (or Auto if your Lab 1D permission rules are in place), then orchestrate:

```
Execute IMPLEMENTATION_PLAN.md as a multi-agent build:
```

1. Spawn a builder subagent to implement the API endpoint and UI section per the plan
 2. In parallel, spawn a test-writer subagent to write tests for the endpoint (happy path, empty data, error handling)
 3. When both finish, run the security-auditor subagent over the changed files
 4. Finally, have the grader subagent score the implementation against rubric.md
- Report each agent's summary and the grader's verdict.

Watch builder and test-writer in the subagent panel while the main session remains responsive. They do not provide guaranteed inline streaming. Assign non-overlapping ownership (`src/` vs `tests/`) or separate worktrees before either writer starts; auditor and grader remain read-only review passes.

Expected output marker: all four agent summaries + a grader table ending `TOTAL: n/10` . If the verdict is REWORK:

```
Fix only the criteria scored below 2, then have the grader re-score.
```

Expected output marker: PASS ($\geq 8/10$).

Run the deterministic tests, clear relevant VS Code Problems, and verify behavior by hand: open `http://localhost:3100` and confirm the Weekly Report section renders. Capture a screenshot or record the exact expected output. Then:

```
/dashboard-review src/routes/
```

Commit:

```
Commit the weekly report feature with a conventional message, mentioning it was built by parallel subagents and graded at [score].
```

Phase 4: Headless CI Check — the Green Light (8 min)

Exit Claude (`exit`). Run your Lab 3C gate against the finished repo:

The script uses the existing Claude Code workstation login. First run `claude -p "Reply with exactly: AUTHENTICATED PRINT MODE READY"` in the VS Code integrated terminal. Do not create or paste an API key.

```
./scripts/ci-review.sh
mkdir -p labs && cp review.json labs/3d-capstone.json
```

Expected output marker:

```
Review score: 8  
Cost: $0.0x  
REVIEW PASSED
```

Exit code 0 = **CI green**. If it fails, read the findings:

```
jq '.structured_output' review.json
```

Start **claude**, paste the top issues with "Fix these findings, then re-run scripts/ci-review.sh", and repeat until green.

Azure-ready handoff: Review **azure-pipelines.yml** and **labs/3c-azure-readiness.md** beside the new capstone diff. Add one sentence to **HANDOFF.md** naming what an authorized Azure owner must validate before enabling Build Validation. Do not run or claim a live Azure pipeline during this lab.

Phase 5: Architecture Decision and Retrospective (5 min)

```
Create `labs/3d-notes.md`. Start with exactly three sentences: (1) the architecture you chose, (2) the alternative you rejected and why, and (3) the escalation trigger that would change your decision. Then, using git history and the current codebase, add a retrospective under 300 words covering what each toolkit artifact adds, how the agents divided today's work, and the session usage/cost evidence (`/usage` for the live subscription session; JSON/provider telemetry for automated runs).
```

Count your prompts across three days. A few dozen messages shipped: a working system, a policy file, two tested skills, enforced hooks, a distributable plugin, an MCP server, a graded multi-agent feature, and a CI pipeline. That's the leverage.

Human Acceptance Gate

The grader does not accept its own build. Before merge, a human reviews the complete Source Control diff, Problems, targeted tests, rendered behavior, local headless evidence, the static Azure-readiness report, and HANDOFF.md. Merge or PR only after that decision.

FINISH MARKER — Capstone Success Criteria (binary)

Complete = ALL THREE true:

- [] **Toolkit repo complete:** CLAUDE.md + skill + hook + command + plugin manifest all PRESENT (Phase 1 gate)
- [] **CI evidence green:** `scripts/ci-review.sh` exits 0 locally; the Azure package remains explicitly unexecuted and ready for an authorized owner
- [] **Decision artifacts saved:** `labs/3d-capstone.json` contains the headless envelope and `labs/3d-notes.md` begins with the required three-sentence architecture decision

Supporting evidence:

- [] Clean baseline commit plus one named capstone worktree/session
- [] IMPLEMENTATION_PLAN.md produced in Plan mode with at least one inline review comment
- [] Feature built by parallel subagents with explicit file ownership, not the main session
- [] Source Control review completed for every changed file; no unexpected files
- [] Relevant VS Code Problems cleared and targeted tests passed
- [] Grader verdict PASS ($\geq 8/10$) with evidence table, followed by a fresh human review
- [] Weekly Report renders in the browser; screenshot or expected output recorded
- [] Conventional commit and HANDOFF.md record the decision and verification evidence
- [] `labs/3d-capstone.json` and `labs/3d-notes.md` are present and reviewed

If Stuck

Problem	Recovery
Phase 1 shows MISSING artifacts	Run that lab's recovery prompt — do not skip the gate
Agents collide on the same file	Re-run with explicit ownership ("builder owns src/, test-writer owns tests/only")
Grader keeps failing one criterion	Read its evidence column — fix that exact line, don't rewrite the feature
ci-review.sh score seems wrong	Check <code>.subtype</code> first, then inspect <code>jq '.structured_output' review.json</code> ; the score follows the validated findings
Ran out of time	Binary criteria first: inventory gate + one green CI run beats a fancy feature
Context bloated	<code>/compact</code> , or rewind to the post-plan checkpoint and re-run Phase 3 tighter

Debrief Questions

1. Which toolkit artifact did the most work during the capstone — policy, skill, hook, command, or plugin?
2. Where did the grader catch something the builder believed was done?
3. What's your week-one plan: which artifact goes into your real repo Monday, and what's your CI entry point?